

— 실습 구성

- 01 Step 0: 환경 진단 + 리포 구조 결정
- 02 Step 1: BE 배포 (Railway) — 8단계
- 03 Step 2: FE 배포 (Vercel) — 5단계
- 04 Step 3: CORS 해결 (자율 디버깅)
- 05 Step 4: 분석 도구 3종 셋업
- 06 Step N: 정리와 비용 비교

— STEP 0

환경 진단과 리포 구조 결정

자료 7 산출물 점검 + 리포 구조 결정 (Railway가 어디를 빌드할지)

- 01 planning_project 폴더에서 작업
- 02 BE 코드(server/) 존재 확인
- 03 리포 구조 결정 (옵션 A vs B)
- 04 BE 코드 process.env.PORT 사용
- 05 .gitignore에 .env* 명시
- 06 /cost 베이스라인 메모

0-1. 환경 확인 명령

운영체제별 명령 — node + git + planning_project 진입

MACOS / LINUX

```
node --version
git --version
cd planning_project
```

WINDOWS (POWERSHELL)

```
node --version
git --version
Set-Location planning_project
```


0-2. 자료 7 산출물 점검

본 학습 진입 전 자료 7 누적 자산 확인

#	산출물	점검
1	화면 2개 동작	카피 입력 + 카피 결과
2	E2E 테스트 통과	e2e/ 폴더
3	scenario-verifier	.claude/agents/ 폴더
4	시나리오 통과 보고	작업자 본인 검증 완료

0-3. 리포 구조 결정 (중요)

본 학습의 리포 구조 두 옵션 — 옵션 A (단일 리포) 권장

옵션	구조	RAILWAY 설정
A: 단일 리포	FE와 BE가 한 리포 (<code>client/</code> , <code>server/</code>)	Root Directory = <code>server/</code>
B: 분리 리포	FE와 BE 별도 리포	Root Directory 설정 불필요

폴더 구조 확인 명령:

ls planning_project/

본 학습은 옵션 A 권장. `planning_project` 하나의 리포로 통합. Railway는 `server/` 만 빌드.

기대 구조:

```
planning_project/
├── src/ ← FE (Next.js)
├── server/ ← BE (Node.js)
│   ├── package.json
│   └── index.js
├── docs/
├── rules/
├── e2e/
├── .claude/
└── .gitignore
```

0-4. BE 코드 점검과 PORT 처리

BE 코드의 포트 처리를 점검. Railway는 **PORT** 환경 변수로 포트 주입.

점검 명령:

```
cat server/index.js | grep -i "port"
```

RAILWAY 호환

```
const port = process.env.PORT || 8000
```

고정 포트 (RAILWAY 빌드 실패 가능)

```
const port = 8000
```

BE 코드가 고정 포트면 Claude Code 명령: "server/index.js의 포트 설정을 **process.env.PORT** 사용으로 변경해줘. 로컬 개발에서는 8000 풀백."

0-5. .gitignore 점검

`.env*` 파일 Git 추적 차단

점검 명령:

```
cat .gitignore | grep ".env"
```

없으면 추가:

```
.env  
.env.local  
.env.production  
node_modules/  
.next/
```

—— 활동 0-6 + 검증 · 토큰 베이스라인 + STEP 0 통과

0-6. 토큰 베이스라인 + Step 0 검증

6개 항목 모두 통과 시 Step 1 (Railway BE 배포) 진입

0-6. 토큰 베이스라인

/cost

Step 0 검증 기준 (6항목)

☐ `planning_project` 폴더에서 작업 중인가

☐ BE 코드(`server/`)가 있는가

☐ 리포 구조가 결정됐는가 (옵션 A 권장)

☐ BE 코드가 `process.env.PORT` 사용하는가

☐ `.gitignore` 에 `.env*` 가 명시됐는가

☐ `/cost` 베이스라인을 메모했는가

— STEP 1

BE 배포 (Railway) — 8단계

Railway에 BE 배포 + 환경 변수 등록 — 클릭 단위로 세분화

- 01 Railway 가입 + GitHub 리포 연결
- 02 Root Directory = server/ 설정
- 03 OPENAI_API_KEY 등록
- 04 Generate Domain → BE URL 발급
- 05 헬스체크 응답 정상
- 06 BE URL 메모 (Step 2-3에서 사용)

1-1. Railway 가입과 첫 접속

메뉴 조작:

1 <https://railway.app> 접속



2 우측 상단 "Login" 클릭



3 "Login with [GitHub](#) " 선택



4 GitHub 권한 승인



5 Railway 대시보드 진입

처음이라면 무료 크레딧 5달러가 자동 부여. 결제 카드 등록 없이 본 학습 진행 가능.

1-2. 새 프로젝트 생성

메뉴 조작:



1-3. Root Directory 설정

반드시 필요한 설정

- 1

프로젝트 화면 → 빌드 중인 Service 클릭
- ↓
- 2

"Settings" 탭 선택
- ↓
- 3

"Build" 섹션의 "Root Directory" 필드 찾기
- ↓
- 4

값 입력: `server`
- ↓
- 5

저장 → 자동 재빌드 트리거

이 설정이 없으면 Railway가 리포의 root(`planning_project/`)를 빌드하려 함 → FE 코드까지 포함되어 실패. `server/` 만 명시해야 BE만 빌드.

1-4. 빌드 명령과 시작 명령 확인

Railway가 자동 감지하지만 점검:

- 1

Settings → "Build" 섹션
- ↓
- 2

Build Command 확인 (Nixpacks가 자동)
`npm install`
- ↓
- 3

Start Command 확인
`npm start`
- ↓
- 4

`server/package.json` 의 "start" 스크립트 점검
`{ "scripts": { "start": "node index.js" } }`

"start" 스크립트가 없으면 Railway 빌드는 성공해도 시작 실패. `package.json` 에 scripts 추가 필요.

1-5. 환경 변수 등록 (OPENAI_API_KEY)

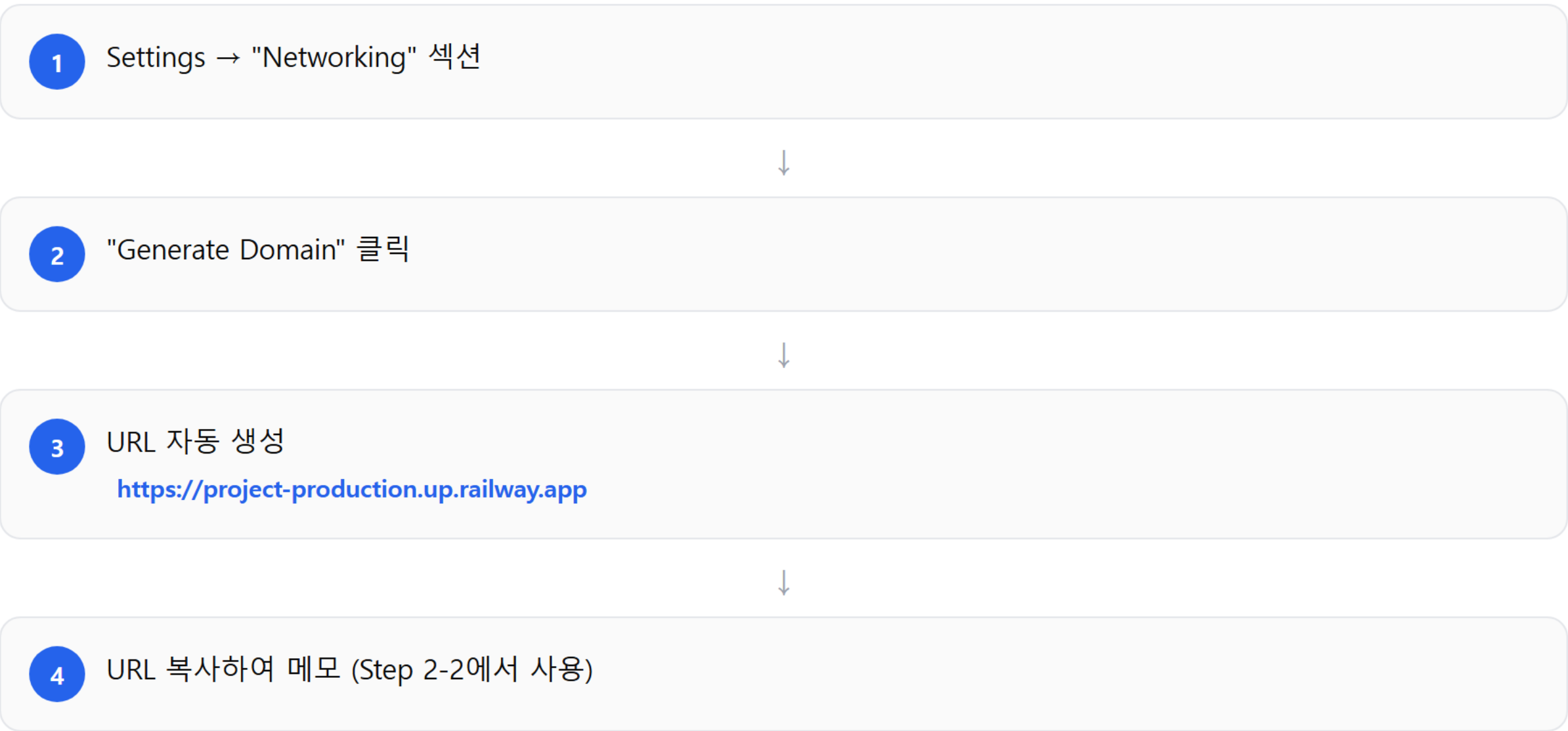
메뉴 조작:



환경 변수 추가 시 Railway가 자동 재배포. 코드 push 없이 변수만 갱신해도 적용.

1-6. Generate Domain (URL 발급)

메뉴 조작:



1-7. 헬스체크

발급된 URL이 실제 응답하는지 확인

MACOS / LINUX

curl https://{Railway URL}/health

WINDOWS

Invoke-WebRequest https://{Railway URL}/health

기대 응답:

```
{"status": "ok"}
```

`/health` 엔드포인트가 BE에 없으면 404 응답. 그러면 Claude Code 명령: "server/에 GET `/health` 엔드포인트 추가해줘. status: ok JSON 응답만."

1-8. 빌드 실패 시 로그 확인

5단계 로그 확인 + 자울 디버깅 3원칙 명령

- 1

Service 화면 → "Deployments" 탭
- 2

실패한 배포 클릭
- 3

"Build Logs" 또는 "Deploy Logs" 확인
- 4

빨간 에러 메시지 풀 텍스트 복사
- 5

자울 디버깅 3원칙 명령
에러 분석 + 수정 방안 요청

흔한 빌드 실패 원인

에러	원인	수정
"Cannot find module"	의존성 누락	<code>server/package.json</code> 에 추가
"Port already in use"	포트 고정	<code>process.env.PORT</code> 사용
"Missing start script"	package.json 누락	" <code>start</code> "스크립트 추가

활동 1-검증 · STEP 1 통과 기준

Step 1 검증 기준

6개 항목 모두 통과 시 Step 2 (Vercel FE 배포) 진입

- ☐ Railway 가입과 GitHub 리포 연결 완료
- ☐ Root Directory가 **server** 로 설정됐는가
- ☐ **OPENAI_API_KEY** 가 Variables에 등록됐는가
- ☐ Generate Domain으로 BE URL 발급됐는가
- ☐ 헬스체크 응답이 정상인가
- ☐ BE URL을 메모했는가

— STEP 2

FE 배포 (Vercel) — 5단계

Vercel에 FE 배포 + Railway BE URL 환경 변수 등록

- 01 Vercel 가입 + GitHub 리포 연결
- 02 Framework Preset 자동 감지 (Next.js)
- 03 NEXT_PUBLIC_API_URL = Railway URL 등록
- 04 FE URL 발급 + 화면 확인 (CORS 에러 예상)

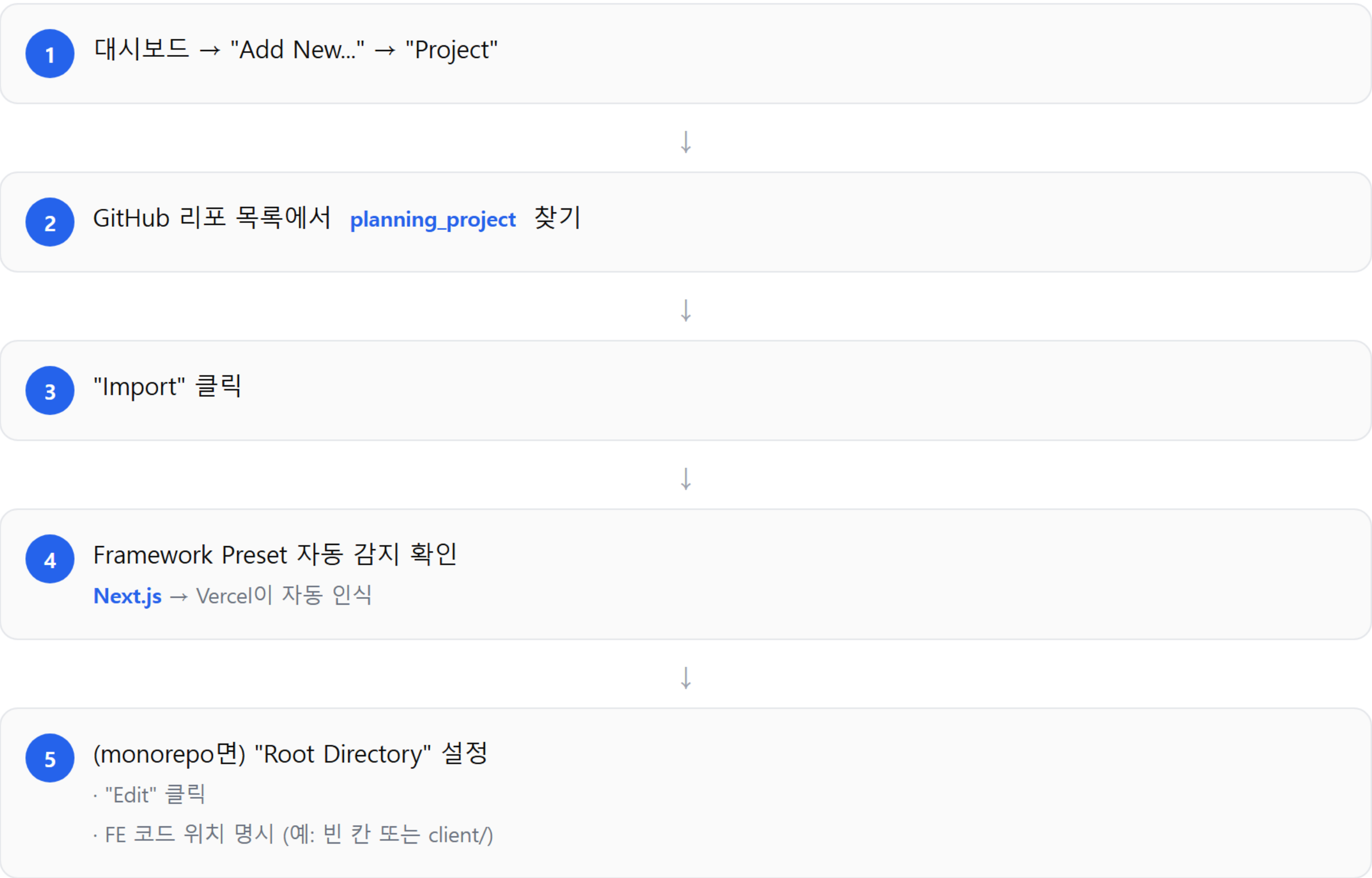
2-1. Vercel 가입과 첫 접속

메뉴 조작:



2-2. Import 화면과 Framework Preset

메뉴 조작:



Vercel 은 Next.js 를 최고 수준으로 통합. Framework Preset이 자동 감지되면 빌드 명령·시작 명령 모두 자동 설정. 직접 건드릴 필요 없음.

2-3. 환경 변수 등록 (NEXT_PUBLIC_API_URL)

Import 화면 또는 배포 후 Settings에서:

1 "Environment Variables" 섹션 펼치기



2 Name 입력: `NEXT_PUBLIC_API_URL`



3 Value 입력: Step 1-6에서 메모한 Railway URL
`https://planning-project-production.up.railway.app`



4 "Add" 클릭

`NEXT_PUBLIC_` 접두사 필수. `Next.js` 가 이 접두사 변수만 브라우저로 노출. 빠뜨리면 FE가 BE URL을 못 읽음.

2-4. 빌드와 URL 발급

메뉴 조작:

1 환경 변수 등록 후 "Deploy" 클릭



2 빌드 로그 화면으로 자동 이동



3 "Building" → "Deploying" → "Ready" 순서로 진행 (2-3분)



4 완료 후 자동 URL 발급
<https://planning-project.vercel.app>



5 "Visit" 또는 "Open" 버튼으로 URL 접속

Vercel 은 매 git push마다 새 URL 자동 발급(preview). main 브랜치 push만 production URL(메인 도메인)로 배포.

2-5. 화면 확인 (CORS 에러 예상)

발급된 Vercel URL을 브라우저로 열기



화면이 뜬 (FE는 정상 동작)



FE가 BE 호출 시도



"blocked by CORS policy" 에러

F12 → Console 탭에서 확인 가능

CORS 에러는 예상된 결과. "배포가 잘됐다는 신호"로 받아들임. 다음 Step에서 해결. **F12** 는 브라우저의 개발자 도구 단축키.

활동 2-검증 · STEP 2 통과 기준

Step 2 검증 기준

4개 항목 모두 통과 시 Step 3 (CORS 해결) 진입

☐ Vercel 가입과 GitHub 리포 연결 완료

☐ FE URL이 발급됐는가

☐ **NEXT_PUBLIC_API_URL** 이 등록됐는가

☐ 화면이 뜨는가 (CORS 에러는 다음 Step)

— STEP 3

CORS 해결 (자율 디버깅 3원칙)

자료 7의 자율 디버깅 3원칙으로 CORS 해결 — 화이트리스트 정책

- 01 CORS 에러 풀 메시지 복사 (F12 Console)
- 02 자율 디버깅 3원칙 명령 사용
- 03 CORS 에러 사라짐 확인
- 04 화이트리스트 정책 (와일드카드 X)
- 05 시드 6번 시나리오 공개 URL 동작

3-1. CORS 에러 풀 메시지 수집

메뉴 조작:



3-1. CORS 에러 메시지 구조

복사한 에러는 보통 이렇게 생김:

```
Access to fetch at 'https://...railway.app/api/generate'
from origin 'https://...vercel.app'
has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present...
```

핵심 정보:

- 어디로 요청: [railway.app](#) (BE)
- 어디서 요청: [vercel.app](#) (FE)
- 왜 차단: [Access-Control-Allow-Origin](#) 헤더 없음

3-2. 자을 디버깅 3원칙 명령

Claude Code에 전달 — 에러 그대로 + 분석 먼저 + 결과 검증

다음 CORS 에러를 분석해주세요:

[에러 풀 메시지 그대로 붙여넣기]

이 BE 코드(server/)에 CORS 설정을 추가해줘.
허용할 FE 도메인은 https://{Vercel URL}.
로컬 개발용 http://localhost:3000도 허용해줘.

설정 후 다른 도메인에서 호출 시 차단되는지도
함께 점검해줘. 수정 후 동작 확인까지 진행해줘.

명령 점검: 3원칙

자율 디버깅 3원칙이 명령에 모두 반영됐는지 확인

원칙	적용 위치
1. 에러 그대로	"에러 풀 메시지 그대로 붙여넣기"
2. 분석 먼저	"분석해주세요" + 수정 명령
3. 결과 검증	"다른 도메인 차단 확인 + 동작 확인"

3-3. 자동 재배포 흐름

자료 4-6의 자동화 인프라가 본 단계에서 작동



자료 4-6에서 만든 자동화 인프라(`git-committer` , hook)가 본 단계에서도 작동. 작업자가 수동 커밋 불필요.

3-4. 동작 확인 + 3-5. CORS 설정 검증

재배포 후 FE에서 시나리오 실행 + 화이트리스트 검증

3-4. 동작 확인

1

시드 6번 시나리오 실행
· 상품 사진 업로드
· 키워드 5개 입력
· "카피 생성" 클릭

2

F12 Console에 CORS 에러 사라졌는지 확인

3

Network 탭에서 BE 호출 200 OK 확인

4

결과 화면에 카피 3개 표시

3-5. CORS 설정 검증

점검 항목	확인
본인 FE 도메인 허용	필수
로컬 개발용 허용	필수
Access-Control-Allow-Origin: * 사용 안 함	필수
알 수 없는 도메인 차단	필수

활동 3-검증 · STEP 3 통과 기준

Step 3 검증 기준

5개 항목 모두 통과 시 Step 4 (분석 도구 3종) 진입

☐ CORS 에러 풀 메시지 복사했는가

☐ 자울 디버깅 3원칙 명령을 사용했는가

☐ CORS 에러가 사라졌는가

☐ 화이트리스트 정책으로 설정됐는가 (와일드카드 X)

☐ 시드 6번 시나리오가 공개 URL에서 동작하는가

— STEP 4

분석 도구 3종 셋업

Clarity + Sentry + 이벤트 추적 — 다음 단계 노출 전 데이터 수집 인프라

- 01 Clarity 추적 코드 layout.tsx 삽입
- 02 Sentry SDK FE+BE 모두 초기화
- 03 이벤트 추적 3개 (페이지/버튼/완료)
- 04 Clarity 대시보드에 본인 세션 1개
- 05 이벤트 3개 Clarity Events 탭
- 06 Sentry 테스트 에러 도착

4-1. Microsoft Clarity 가입과 추적 코드

메뉴 조작:

1

clarity.microsoft.com 접속

2

"Sign up" → Microsoft 계정으로 로그인

3

대시보드 → "+ New Project" 클릭

4

입력

- Name: planning_project
- URL: vercel-url
- Category: Other

5

"Create" 클릭

6

"Settings" → "Setup" 탭

7

"Tracking code" 스크립트 복사
(clarity.start로 시작하는 약 10줄)



4-1. Clarity 코드 삽입 명령

Claude Code에 전달 — Next.js의 `src/app/layout.tsx` 에 삽입

프로젝트 FE에 Microsoft Clarity 추적 코드를 삽입해줘.

Clarity 코드는 다음과 같습니다:
[추적 코드 그대로 붙여넣기]

Next.js의 `src/app/layout.tsx`에 삽입해서
모든 페이지에 자동 로드되도록 설정해줘.

설치 후 Clarity 대시보드에 데이터 도착 여부를
어떻게 확인하는지도 알려줘.

4-2. Sentry 가입과 DSN 발급

메뉴 조작 — FE/BE 별도 프로젝트 2개

1

sentry.io 접속 → "Sign Up"

2

무료 계정 생성 (GitHub OAuth 가능)

3

"+ Create Project" 클릭

4

Platform 선택
· FE: Next.js
· BE: Node.js

5

Project name 입력
예: **planning-project-fe**

6

"Create Project" → **DSN** 자동 발급

7

DSN 복사
https://...@sentry.io/...

8

BE용도 별도 프로젝트로 생성해 **DSN** 받기

FE **DSN** 과 BE **DSN** 은 다른 값. 두 프로젝트 별도 생성. 무료 티어는 월 5,000 에러 이벤트 제공.

4-2. Sentry SDK 설치 명령

Claude Code에 전달 — FE/BE 양쪽 초기화

프로젝트 FE와 BE에 Sentry SDK를 설치하고 초기화해줘.

Sentry DSN:

- FE: [FE DSN 그대로 붙여넣기]
- BE: [BE DSN 그대로 붙여넣기]

에러 발생 시 다음 컨텍스트도 전송하도록 설정:

- 사용자가 있던 페이지 URL
- 클릭한 버튼이나 입력값 (개인정보 제외)
- BE의 경우 어느 API 엔드포인트인지

테스트 에러를 한 번 발생시켜 Sentry 대시보드에 도착하는지도 확인해줘.

4-3. 이벤트 추적 코드 결정

시드 6번 핵심 이벤트 3개

이벤트	시드 6번	위치
페이지 진입	카피 입력 화면 방문	<code>InputForm.tsx</code>
핵심 버튼 클릭	"카피 생성" 클릭	InputForm 버튼
핵심 기능 완료	카피 1개 복사	<code>CopyButton.tsx</code>

4-3. 이벤트 추적 코드 삽입 명령

Claude Code에 전달 — `window.clarity("set", "event_name", "value")` 코드 삽입

docs/prd.md의 핵심 시나리오에서 추적할 이벤트 3개를 결정해줘.

기준:

- 1. 페이지 진입 (자동으로 잡힘)
- 2. 핵심 버튼 클릭 (시드 6번 기준 "카피 생성")
- 3. 핵심 기능 완료 (시드 6번 기준 "카피 복사")

각 이벤트에 대해 Microsoft Clarity의 custom event 추적 코드 (`window.clarity("set", "event_name", "value")`)를 어디에 삽입해야 할지 분석한 뒤 코드를 추가해줘.

이벤트명은 PRD 가설과 연결 가능하게 명명해줘 (예: `copy_generation_completed`).

활동 4-4 · 본인 시나리오 실행

4-4. 본인이 시나리오 실행

셋업 직후 시나리오 1회 실행 + 대시보드 확인

1

공개 URL 접속 (Vercel URL)

2

시드 6번 시나리오 1회 실행

- 사진 업로드
- 키워드 5개
- 카피 생성
- 결과 복사

3

Clarity 대시보드 (5-10분 후)
→ 세션 1개 보이는지

4

Sentry 대시보드 확인
→ 의도적 에러 도착

5

Clarity → Events 탭
→ 이벤트 3개 기록

세션이 대시보드에 보이기까지 5-10분 지연. 즉시 안 보여도 정상. 셋업 직후 시나리오 1회 실행 후 내일 아침에 확인하는 흐름도 가능.

활동 4-검증 · STEP 4 통과 기준

Step 4 검증 기준

6개 항목 모두 통과 시 Step N (정리·회고) 진입

- ☐ Clarity 추적 코드가 `layout.tsx` 에 삽입됐는가
- ☐ Sentry SDK가 FE와 BE 모두 초기화됐는가
- ☐ 이벤트 추적 코드 3개가 적절한 위치에 있는가
- ☐ Clarity 대시보드에 본인 세션 1개가 들어왔는가
- ☐ 이벤트 3개가 Clarity Events 탭에 기록됐는가
- ☐ Sentry 테스트 에러가 도착하는가

— STEP N

정리와 비용 비교

산출물 6개 확인 + /cost 비교 + 회고

- 01 산출물 6개 정리 (FE+BE URL + CORS + Clarity + Sentry + 이벤트)
- 02 토큰 비용 비교 (/cost)
- 03 회고 3 질문 (좋은 답 vs 나쁜 답)
- 04 다음 단계(노출) 진입 준비 확인

N-1. 산출물 정리

본 학습의 6개 산출물 + 다음 단계 활용

#	산출물	다음 단계 활용
1	공개 FE URL (Vercel)	사용자 노출
2	공개 BE URL (Railway)	API 처리
3	CORS 해결과 환경 변수 셋업	안정 운영
4	Microsoft Clarity 추적	UX 가설 검증
5	Sentry 에러 트래킹	품질 안전망
6	이벤트 추적 코드 3개	PRD 가설 검증

활동 N-2 · 비용 비교

N-2. /cost 비교

본 sprint 종료 시 토큰 사용량 비교

/cost

Step 0 베이스라인과 비교 — 본 단계 누적 토큰 사용량 메모. 본 학습 후 본격 운영 시 비용 통제 결정 근거.

N-3. 회고: 좋은 답 vs 나쁜 답

구체적·근거 있는 답이 좋은 답

질문	좋은 답	나쁜 답
개발 어휘 효과	<code>PORT</code> , <code>CORS</code> , <code>NEXT_PUBLIC_</code> 어휘 알고 나니 막힐 때 정확한 명령	"어렵지 않았다"
CORS 자을 디버깅 효과	풀 메시지 그대로 → 화이트리스트 설정 명령 → 1회 해결	"디버깅 됐다"
분석 도구 가치	<code>Clarity</code> 는 UX 막힘, <code>Sentry</code> 는 모르는 에러, 이벤트는 PRD 검증. 셋이 서로 다른 질문에 답함	"다 같았다"

활동 N-검증 · STEP N 통과 기준

Step N 검증 기준

4개 항목 모두 통과 시 본 sprint 완료 + 다음 단계(노출) 진입 가능

- ☐ 산출물 6개가 모두 손에 있는가
- ☐ `/cost` 비교를 메모했는가
- ☐ 회고 3개 질문에 모두 답했는가
- ☐ 다음 단계(노출) 진입 준비가 됐는가

과제

본 자료의 과제는 `assignment.md` 파일에 정리되어 있다.

항목	내용
목표	분석 인프라 + 카피 준비, 배포 URL 안정화
산출물	<code>Clarity</code> 세션 1+, <code>Sentry</code> 연결, 이벤트 3개, 재피드백 메일 카피
마감	다음 학습 단계 시작 전

공개 URL과 분석 인프라가 갖춰진 상태가 다음 단계(노출) 출발선.

— TIP 1

최소 개발 지식 팁

개발 어휘와 F12

최소 개발 지식 팁

어휘가 명령의 정확성을 결정

`PORT` , `CORS` , `NEXT_PUBLIC_` 같은 어휘를 알면 AI 명령이 정확. 모르면 추측 명령으로 결과 어긋남.

F12는 모든 디버깅의 출발

브라우저 개발자 도구. Console 탭에서 에러 메시지 복사하는 동작이 표준.

— TIP 2

분리 배포 팁

신뢰 경계 + 키 노출 대응

분리 배포 팁

신뢰 경계 기억

FE는 신뢰 X, BE는 신뢰 O. 비밀은 BE에만. `NEXT_PUBLIC_` 접두사 변수에 비밀 두면 그 즉시 노출.

API 키 노출 시 즉시 회수

GitHub 푸시 후 봇이 수초 안에 발견. 키 삭제 → 새 발급 → git 히스토리 정리.

— TIP 3

Railway **팁**

Root Directory + PORT + 무료 티어

Railway 팁

Root Directory 설정 중요

monorepo면 `server/` 명시. 안 하면 FE까지 함께 빌드되어 실패.

PORT 환경 변수

BE 코드는 `process.env.PORT` 사용. 고정 포트면 빌드 실패.

무료 티어 영원하지 않음

월 5달러 크레딧. 본격 운영 시 결제 필요할 수 있음.

— TIP 4

Vercel 팁

Framework Preset + 자동 재배포

Vercel 팁

Framework Preset 자동 감지

Next.js는 자동. 빌드 명령 직접 건드릴 필요 없음.

환경 변수 변경 시 재배포

추가 후 자동 재배포 트리거. 코드 푸시 불필요.

— TIP 5

CORS 팁

통과 의례 + 화이트리스트 + 자율 디버깅

CORS 팁

첫 만남은 통과 의례

처음 배포 후 무조건 한 번 막힘. 정상. "배포가 잘됐다는 신호"로 받아들임.

화이트리스트 정책 고수

Access-Control-Allow-Origin: * 사용 금지. 본인 FE 도메인만 명시.

자율 디버깅 3원칙 그대로 적용

자료 7의 패턴: 풀 메시지 그대로 → 분석 먼저 → 결과 검증.

— TIP 6

분석 도구 팁

셋업 시점 + 3 도구의 다른 질문 + 이벤트명

분석 도구 팁

사용자 들어오기 전에 깎다

들어온 후 깔면 그 사용자의 행동은 영원히 못 본다. 1명의 세션은 한 번뿐.

세 도구의 다른 질문 기억

도구	질문
Clarity	어디서 막혔는가
Sentry	무슨 에러 있는가
이벤트 추적	가설 통과하는가

한 도구로 다 해결 안 됨.

이벤트 추적은 3개로 시작

페이지 진입 / 핵심 버튼 / 핵심 기능 완료. 나머지는 자가 학습 영역.

이벤트명은 PRD 가설과 연결

`copy_generation_completed` 같은 명명. 어떤 가설 검증과 연결되는지 명확하게.

— TIP 7

의사결정 흐름 팁

세 도구 데이터 종합 — 다른 의사결정 영역

의사결정 흐름 팁

세 도구 데이터가 종합되어야 결정

- **Sentry** → 버그 수정
- **Clarity** → UX 개선
- **이벤트 추적** → PRD 갱신

각각 다른 의사결정 영역. 한 보고로 모든 결정 X.